



Chapter 4

Accessing Data in Files

IN THIS CHAPTER

» Working with simple and formatted text data

» Using alternative flat-file sources

» Considering nontext sources

» Interacting with online data

.....

Unless the data you use is a live feed or is generated in some manner, it likely comes from a file. In fact, files provide the means to preserve the majority of data used for analysis purposes. So the use of files for storage isn't the issue; instead, the question is what sort of file you might expect to see. The format of the data in the file says a lot about the kind of information you can expect from it. [Chapter 1](#) of this mini-book describes the format of files from an overview perspective. This chapter demonstrates how to access data found in files of various formats.

Major divisions appear in the complexity of file storage as well. The files described in this chapter are flat file in nature, which means that they contain just one table. However, the chapter also discusses both textual and binary storage in both local and online sources. [Chapter 5](#)

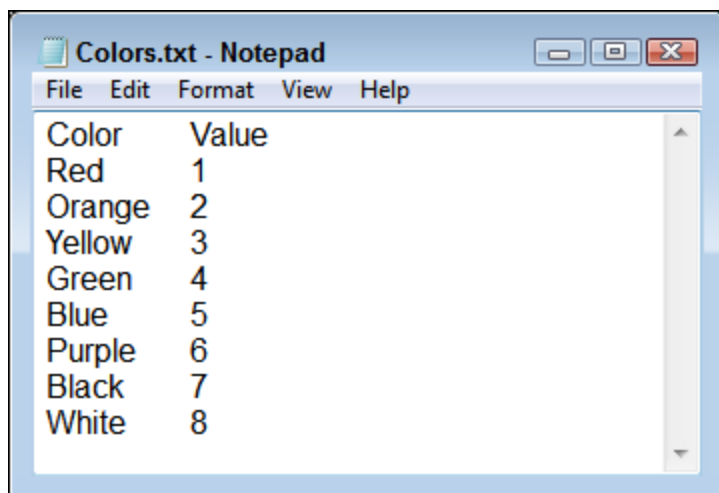
of this minibook looks at relational databases, and [Chapter 6](#) discusses NoSQL databases. Each form of data storage has pros and cons.



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0204_Data.ipynb` source code file for Python and the `DSPD_R_0204_Data.ipynb` source code file for R. See the Introduction for details on how to find these source files. In addition to the source code files, you must also have access to the `Colors.txt`, `FixedWidth.txt`, `FixedWidth2.txt`, `Titanic.csv`, `Colors.csv`, `XMLData.xml`, and `Values.xls` files.

Understanding Flat File Data Sources

The most commonly used flat file is a pure text file because you can send it nearly anywhere. Text files can use a variety of storage formats. However, a common format is to have a header line that documents the purpose of each field, followed by another line for each record in the file. The file separates the fields using tabs. See [Figure 4-1](#) for an example of the `Colors.txt` file used for the example in this section.



Color	Value
Red	1
Orange	2
Yellow	3
Green	4
Blue	5
Purple	6
Black	7
White	8

FIGURE 4-1: A text file contains only text and a little formatting with control characters.

Native Python provides a wide variety of methods that you can use to read such a file. However, letting someone else do the work is far easier. In this case, you can use the `pandas` library to perform the task. Within the `pandas` library, you find a set of *parsers*, code used to read individual bits of data and determine the purpose of each bit according to the format of the entire file. Using the correct parser is essential if you want to make sense of file content. In this case, you use the `read_csv()` method to accomplish the task (older versions of `pandas` use the `read_table()` method instead), as shown in the following code:

```
import pandas as pd

color_table = pd.io.parsers.read_csv("Colors.txt",

                                     sep='\t')

print(color_table)
```

**TIP**

The code imports the `pandas` library, uses the `read_csv()` method to read `Colors.txt` into a variable named `color_table`, and then displays the resulting memory data onscreen using the `print` function. Note that you must use the `sep='\t'` argument because the text file uses tabs to separate field entries. Here's the output you can expect to see from this example:

	Color	Value
0	Red	1
1	Orange	2
2	Yellow	3

3	Green	4	
4	Blue	5	
5	Purple	6	
6	Black	7	
7	White	8	

Notice that the parser correctly interprets the first row as consisting of field names. It numbers the records from 0 through 7. Using `read_csv()` method arguments, you can adjust how the parser interprets the input file, but the default settings usually work best. You can read more about the `read_csv()` arguments at https://pandas.pydata.org/pandas-docs/version/0.25.0/reference/api/pandas.read_csv.html.

Working with Positional Data Files

Positional data files, those in which each field takes up a specific amount of space, see use in various technologies, such as Point-of-Sale (POS) systems. These files also go by the names fixed width-field and fixed-length files. No matter what you call them, an advantage of this file format is that you can read the data as a record or a structure in most programming languages. Processing is simplified because you don't have to worry about any (or very few) control characters. The downside is that these files consume more space than flat files that rely on tabs, but even the space requirement is easier to calculate because each record will always consume the same amount of space.

You usually find positional data files in two forms. The first eschews any sort of control characters at all, and the second relies on a carriage return, linefeed, or both to separate records. The files lack other control characters and you generally see them limited to the standard

(7-bit) or extended (8-bit) ASCII character set. At least one vendor relied on a custom 6-bit (64-character) setup to save space (by using uppercase alpha characters, ten digits, and a number of punctuation marks such as period, question mark, space, and comma). With these constraints in mind, [Figure 4-2](#) shows the `FixedWidth.txt` file used for the first example in this section.

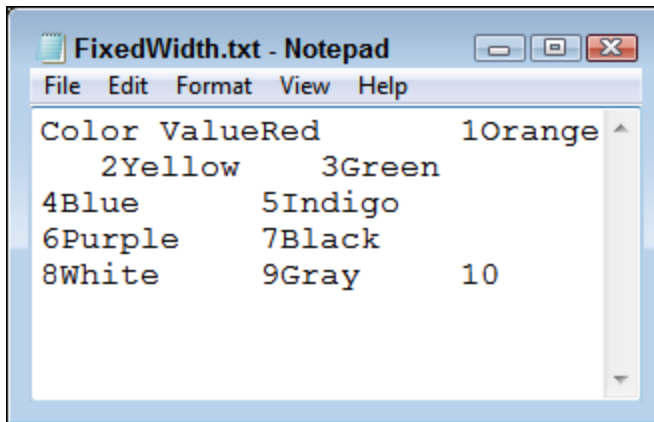


FIGURE 4-2: Each field in this file consumes precisely the same space.

Notice that no formatting is provided, as shown in [Figure 4-1](#). All the data simply runs together. You use a different technique to process this file than you do with a flat file, as shown here:

```
row = 0
```

```
with open("FixedWidth.txt", "r") as FW:
```

```
    while True:
```

```
        Color = FW.read(6)
```

```
        if not Color:
```

```
            break
```

```
        Value = FW.read(5)
```

```
if row == 0:
```

```
    color_table2 = pd.DataFrame(
```

```
        columns=[Color, Value])
```

```
else:
```

```
    color_table2 = color_table2.append(
```

```
        [{'Color ': Color, 'Value': Value}],
```

```
        ignore_index=True, sort=False)
```

```
    row=row+1
```

```
print(color_table2)
```

The lack of any sort of row marker means that you must process this file record by record rather than line by line. The first entry is the column header, so the code produces a `DataFrame` containing the `columns` it finds. The code assumes that it's reading a particular file. You could modify the code to make it more generic and use it for any file. Each of the rows that follow is appended to the `DataFrame`, as shown. Here's what you see as output:

	Color	Value
--	-------	-------

0	Red	1
---	-----	---

1	Orange	2
---	--------	---

2	Yellow	3
---	--------	---

3	Green	4
4	Blue	5
5	Indigo	6
6	Purple	7
7	Black	8
8	White	9
9	Gray	10

As you can see, the resulting `DataFrame` provides precisely what you expect in terms of formatted data. However, you may find that your positional data file does provide some sort of row marker, as shown in [Figure 4-3](#).



FIGURE 4-3: This file includes carriage returns for row indicators.

In this case, processing the file becomes infinitely easier, as shown here:

```
color_table3 = pd.read_fwf("FixedWidth2.txt",
```

```
widths=[6, 5])
```

```
print(color_table3)
```

The output is the same as before, but the amount of processing you need to perform is significantly less. The `read_fwf()` function assumes that the first row is a header, as described at https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_fwf.html. If your file lacks a header, you simply add the `header=None` argument. You can use the other arguments defined for a `TextFileWriter` at <https://tedboy.github.io/pandas/io/io2.html> as well.

Accessing Data in CSV Files

The Comma-Separated Value (CSV) format predates the PC by a number of years. In fact, IBM originally used it with the Formula Translation (FORTRAN) language, as described at <https://blog.sqlizer.io/posts/csv-history/>. The fact that CSV is so obvious and useful at the same time explains why it has remained one of the better methods to transfer data from anyplace to anywhere. The following sections tell you more about CSV and explain how you can use it in your data science endeavors.

Working with a simple CSV file

One of the things that differentiates a database from a pure text file is formatting, which enables you to discover more about the data simply by the manner in which it's organized. Of the simple database files, CSV is the most common because you can send it almost anywhere and it doesn't consume a lot of space. A CSV file also provides more formatting than a simple text file. In fact, CSV files can become quite complicated. There is a standard that defines the format of CSV files, and you can see it at <https://tools.ietf.org/html/rfc4180>. The CSV file used for this example is quite simple:

- A header defines each of the fields
- Fields are separated by commas
- Records are separated by linefeeds
- Strings are enclosed in double quotes
- Integers and real numbers appear without double quotes

[Figure 4-4](#) shows the raw format for the `Titanic.csv` file used for this example. You can see the raw format using any text editor.

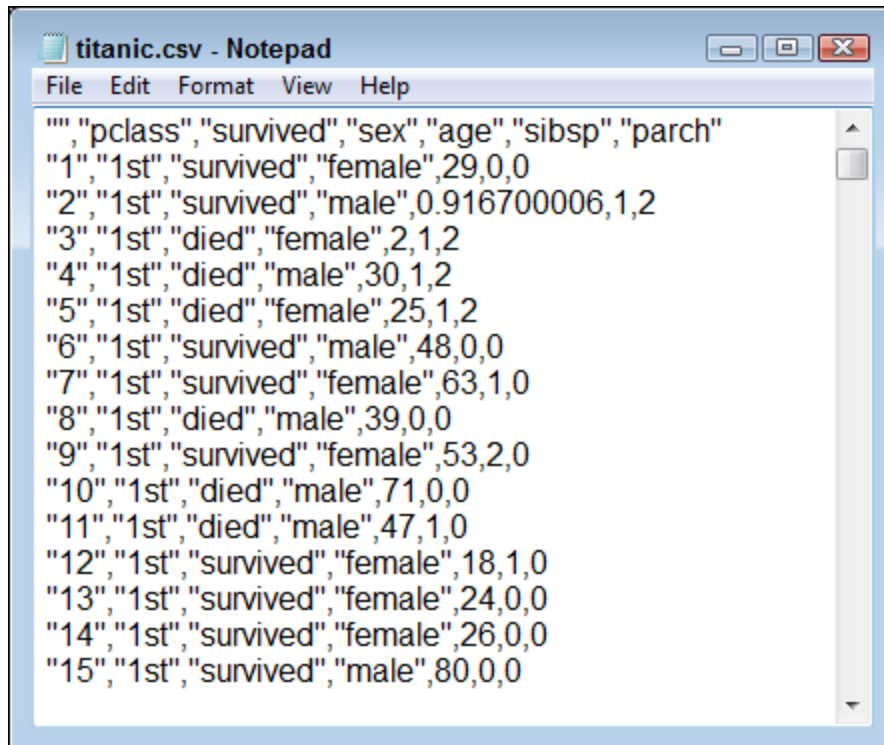


FIGURE 4-4: The raw format of a CSV file is still text and quite readable.

Applications such as Excel can import and format CSV files so that they become easier to read. [Figure 4-5](#) shows the same file in Excel.

Excel actually recognizes the header as a header. If you were to use features such as data sorting, you could select header columns to obtain the desired result. Fortunately, `pandas` also enables you to work with the CSV file as formatted data, as shown in the following example:

```
import pandas as pd
```

```
titanic = pd.read_csv("Titanic.csv")
```

```
X = titanic[['age']]
```

```
print(X)
```

	A	B	C	D	E	F	G	H
1		1st	survived	female	29	0	0	
2		2nd	survived	male	0.9167	1	2	
3		3rd	died	female	2	1	2	
4		4th	died	male	30	1	2	
5		5th	died	female	25	1	2	
6		6th	survived	male	48	0	0	

FIGURE 4-5: Use an application such as Excel to create a formatted CSV presentation.

As with the previous example, this example uses the `read_csv()` parser. However, because a CSV file provides more formatting information, you don't need to add the `sep='\t'` argument. Selecting a specific field is quite easy — you just supply the field name as shown. The output from this example looks like this (although some values are omitted for the sake of space):

```
age
```

```
0    29.0000
```

```
1    0.9167
```

```
2    2.0000
```

```
3   30.0000
```

```

4      25.0000
5      48.0000
...
1304   14.5000
1305  9999.0000
1306   26.5000
1307   27.0000
1308   29.0000

[1309 rows x 1 columns]

```

**TIP**

A human-readable output like this one is nice when working through an example, but you might also need the output as a list. To create the output as a list, you simply change the third line of code to read `X = titanic[['age']].values`. Notice the addition of the `values` property. The output changes to something like this (with some values omitted for the sake of space):

```

[[ 29. ]
 [ 0.91670001]
 [ 2. ]
 ...,
 [ 26.5 ]

```

```
[ 27. ]
```

```
[ 29. ]]
```

Making use of header information

The previous section makes use of a standards-based CSV file, which doesn't offer much in the way of data format information. If you know the file well through examination, you can always guess about its content, but that's not the same as actually knowing the data types. Some CSV users extend the standard by adding header information as described at <https://support.spatialkey.com/providing-data-types-in-csv-headers/> and

<https://support.symantec.com/us/en/article.HOWTO41097.html>.

Figure 4-6 shows a CSV file that employs some of these concepts to provide type information.

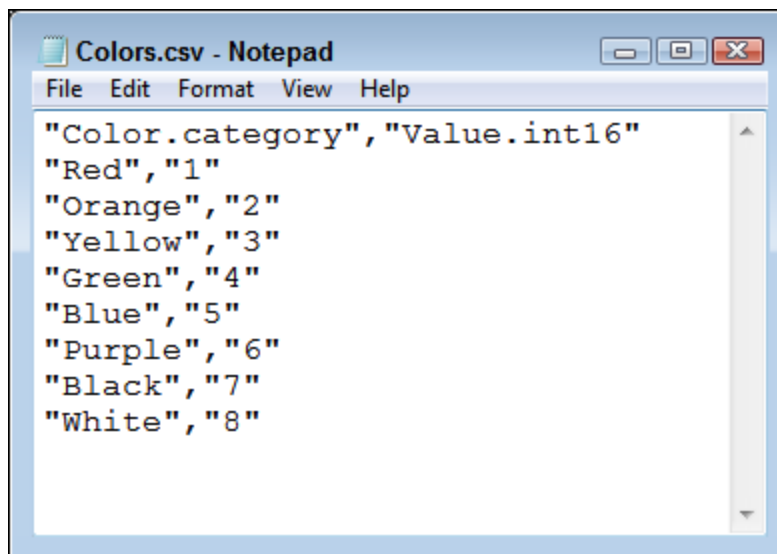


FIGURE 4-6: CSV headers can contain data type information, among other clues.

Obviously, processing this file requires a little extra code, as shown here:

```
color_table4 = pd.read_csv("Colors.csv")
```

```
print("Original types:\n", color_table4.dtypes, "\n")
```

```
for col in color_table4.columns:
```

```
    col_split = col.split('.')

```

```
    color_table4 = color_table4.rename(

```

```
        columns={col: col_split[0]})

```

```
    color_table4 = color_table4.astype(

```

```
        {col_split[0]: col_split[1]})

```

```
print("New types:\n", color_table4.dtypes, "\n")

```

```
print(color_table4)

```

The header information now contains both a header name and a data type. The first `print` statement shows you the original header state, including the data types it uses. The `for` loop removes the type information from the header and uses it to change the data type of each column. Here is the output from this example:

```
Original types:

```

```
Color.category    object

```

```
Value.int16      int64

```

```
dtype: object

```

```
New types:

```

Color	category	
Value	int16	
dtype:	object	
Color	Value	
0	Red	1
1	Orange	2
2	Yellow	3
3	Green	4
4	Blue	5
5	Purple	6
6	Black	7
7	White	8

Moving On to XML Files

Markup languages as a whole have been around for a number of years. The XML file format started as a means to transfer formatted data when scientists found that HTML just wasn't up to the task. You can read a brief history about XML at

<https://ccollins.wordpress.com/2008/03/03/a-brief-history-of-xml/>. XML provides metadata that you won't find with a flat file.

This metadata helps preserve the format and context of data. The following sections tell you more about XML and describe how to use it.

Working with a simple XML file

One of the most beneficial data access techniques to know when working with web data is accessing XML. All sorts of content types, and even some web pages, rely on XML. Working with web services and microservices means working with XML (in most cases). With this idea in mind, the example in this section works with XML data found in the `XMLData.xml` file, shown in [Figure 4-7](#). In this case, the file is simple and uses only a couple of levels. XML is hierarchical and can become quite a few levels deep.

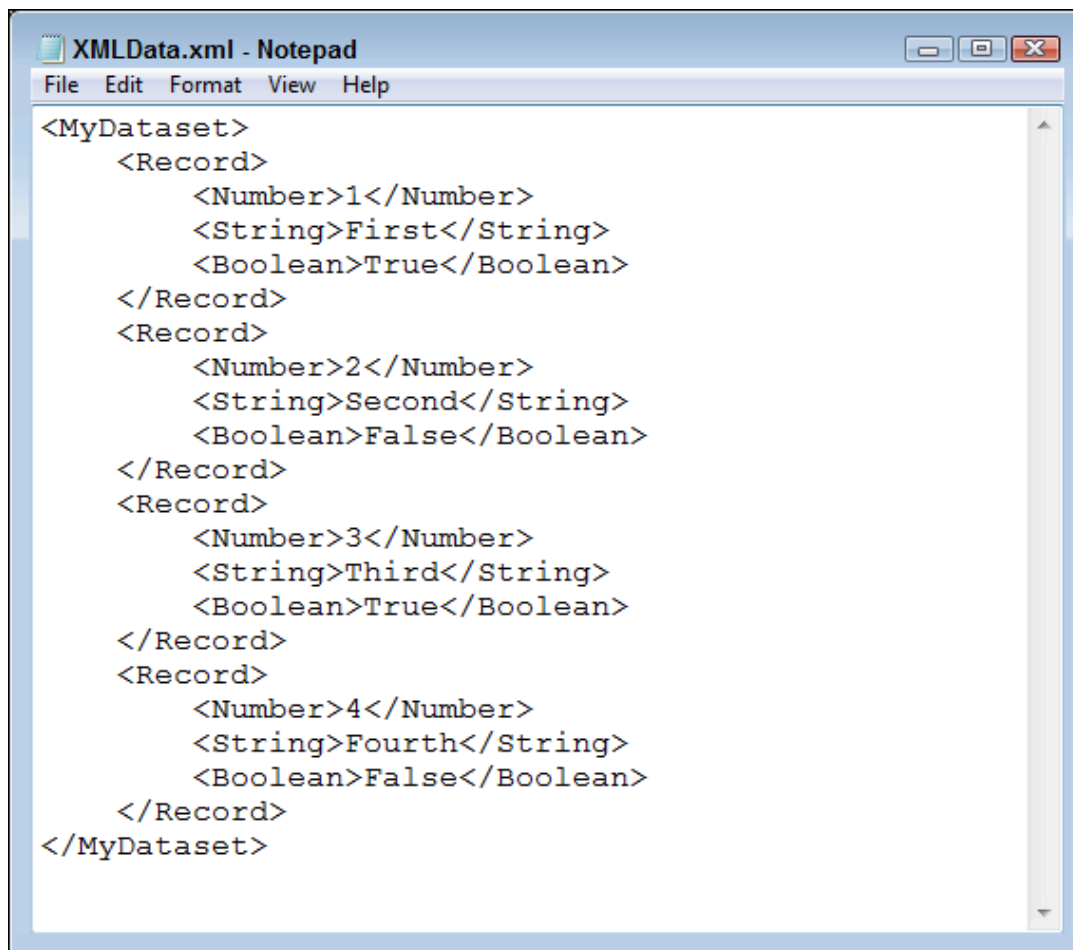


FIGURE 4-7: XML is a hierarchical format that can become quite complex.

The technique for working with XML, even simple XML, can be a bit harder than anything else you've worked with so far. Here's the code for this example:

```
from lxml import objectify

import pandas as pd

xml = objectify.parse(open('XMLData.xml'))

root = xml.getroot()

df = pd.DataFrame(columns=('Number', 'String', 'Boolean'))

for i in range(0,4):

    obj = root.getchildren()[i].getchildren()

    row = dict(zip(['Number', 'String', 'Boolean'],

                   [obj[0].text, obj[1].text,

                    obj[2].text]))

    row_s = pd.Series(row)

    row_s.name = i

    df = df.append(row_s)

print(df)
```


The example begins by importing libraries and parsing the data file using the `objectify.parse()` method. Every XML document must contain a root node, which is `<MyDataset>` in this case. The root node encapsulates the rest of the content, and every node under it is a child. To do anything practical with the document, you must obtain access to the root node using the `getroot()` method.

The next step is to create an empty `DataFrame` object that contains the correct column names for each record entry: `Number`, `String`, and `Boolean`. As with all other pandas data handling, XML data handling relies on a `DataFrame`. The `for` loop fills the `DataFrame` with the four records from the XML file (each in a `<Record>` node).

The process looks complex but follows a logical order. The `obj` variable contains all the children for one `<Record>` node. These children are loaded into a dictionary object in which the keys are `Number`, `String`, and `Boolean` to match the `DataFrame` columns.

Now the code has a dictionary object that contains the row data. The code creates an actual row for the `DataFrame` next. It gives the row the value of the current `for` loop iteration. It then appends the row to the `DataFrame`. To see that everything worked as expected, the code prints the result, which looks like this:

	Number	String	Boolean
0	1	First	True
1	2	Second	False
2	3	Third	True
3	4	Fourth	False

Parsing XML

XML files provide additional information about the content of a dataset, but they aren't perfect. Simply extracting data from an XML file may not be enough to use it because the data may not be in the correct format. You could end up with a dataset containing only strings. Obviously, you can't perform much data manipulation with strings. The following example reads an XML file into memory and shapes the XML data to create a new `DataFrame` containing just the `<Number>` and `<Boolean>` elements in the correct format:

```
from lxml import objectify

import pandas as pd

from distutils import util


xml = objectify.parse(open('XMLData.xml'))

root = xml.getroot()

df = pd.DataFrame(columns=('Number', 'Boolean'))


for i in range(0, 4):

    obj = root.getchildren()[i].getchildren()

    row = dict(zip(['Number', 'Boolean'],

                   [obj[0].pyval,

                    bool(util strtobool(obj[2].text))]))

    row_s = pd.Series(row)
```

```
row_s.name = obj[1].text
```

```
df = df.append(row_s)
```

```
print(type(df.loc['First']['Number']))
```

```
print(type(df.loc['First']['Boolean']))
```

The DataFrame `df` is initially instantiated as empty, but as the code loops through the root node's children, it extracts a list containing the following:

- A `<Number>` element (expressed as an `int`)
- An ordinal element (a `string`)
- A `<Boolean>` element (expressed as a `string`)

that the code uses to increment `df`. In fact, the code relies on the ordinal number element as the index label and constructs a new individual row to append to the existing DataFrame. This operation programmatically converts the information contained in the XML tree into the right data type to place into the existing variables in `df`. The number elements are already available as `int` type; the conversion of the `<Boolean>` element is a little harder. You must convert the string to a numeric value using the `strtobool()` function in `distutils.util`. The output is a `0` for `False` values and a `1` for `True` values. However, that's still not a Boolean value. To create a Boolean value, you must convert the 0 or 1 using `bool()`.



TIP

This example also shows how to access individual values in the DataFrame. Notice that the `name` property now uses the `<String>` element value for easy access. You provide an index value

using `loc` and then access the individual feature using a second index. The output from this example is

```
<class 'int'>
```

```
<class 'bool'>
```

Using XPath for data extraction

Using XPath to extract data from your dataset can greatly reduce the complexity of your code and potentially make it faster as well. The following example shows an XPath version of the example in the previous section. Notice that this version is shorter and doesn't require the use of a `for` loop.

```
from lxml import objectify
```

```
import pandas as pd
```

```
from distutils import util
```

```
xml = objectify.parse(open('XMLData.xml'))
```

```
root = xml.getroot()
```

```
map_number = map(int, root.xpath('Record/Number'))
```

```
map_bool = map(str, root.xpath('Record/Boolean'))
```

```
map_bool = map(util.strtobool, map_bool)
```

```
map_bool = map(bool, map_bool)
```

```
map_string = map(str, root.xpath('Record/String'))
```

```
data = list(zip(map_number, map_bool))
```

```
df = pd.DataFrame(data,
```

```
columns=('Number', 'Boolean'),
```

```
index = list(map_string))
```

```
print(df)
```

```
print(type(df.loc['First']['Number']))
```

```
print(type(df.loc['First']['Boolean']))
```

The example begins just like the previous example, with the importing of data and obtaining of the root node. At this point, the example creates a data object that contains record number and Boolean value pairs. Because the XML file entries are all strings, you must use the `map()` function to convert the strings to the appropriate values.

Working with the record number is straightforward: You just map it to an `int`. The `xpath()` function accepts a path from the root node to the data you need, which is `'Record/Number'` in this case.

Mapping the Boolean value is a little more difficult. As in the previous section, you must use the `util.strtobool()` function to convert the string Boolean values to a number that `bool()` can convert to a Boolean equivalent. However, if you try to perform just a double mapping, you'll encounter an error message saying that lists don't include

a required function, `tolower()`. To overcome this obstacle, you perform a triple mapping and convert the data to a string using the `str()` function first.

Creating the `DataFrame` is different, too. Instead of adding individual rows, you add all the rows at one time by using `data`. Setting up the column names is the same as before. However, now you need some way of adding the row names, as in the previous example. This task is accomplished by setting the `index` parameter to a mapped version of the `xpath()` output for the `'Record/String'` path. Here's the output you can expect:

Number Boolean			
First	1	True	
Second	2	False	
Third	3	True	
Fourth	4	False	
<type 'numpy.int64'>			
<type 'numpy.bool_'>			

Considering Other Flat-File Data Sources

You have many other flat-file data sources at your disposal, all of which likely require specialized processing. This chapter uses Excel, an application that you might have on your system. If you don't have the application on your system, you can interact with an Internet version of the application. Excel and other Microsoft Office applications provide highly formatted content that you can use in a freeform database manner. You can specify every aspect of the information these

files contain. The `Values.xls` file used for this example provides a listing of sine, cosine, and tangent values for a random list of angles. You can see this file in [Figure 4-8](#).

	A	B	C	D	E
1	Angle (Degrees)	Sine	Cosine	Tangent	
2	40.29472	0.646719	0.762728	0.847903	
3	216.71810	-0.597878	-0.801587	0.745868	
4	105.17861	0.965114	-0.261829	-3.686049	
5	97.38824	0.991698	-0.128592	-7.711971	
6	120.87683	0.858272	-0.513194	-1.672413	
7	316.08650	-0.693572	0.720388	-0.962775	
8	317.88761	-0.670587	0.741831	-0.903962	
9	60.82377	0.873124	0.487497	1.791034	
10	34.41988	0.565253	0.824917	0.685224	
11	92.81788	0.998791	-0.049161	-20.316545	

FIGURE 4-8: An Excel file is highly formatted and might contain information of various types.

When you work with Excel or other Microsoft Office products, you begin to experience some complexity. For example, an Excel file can contain more than one worksheet, so you need to tell `pandas` which worksheet to process. In fact, you can choose to process multiple worksheets, if desired. When working with other Office products, you have to be specific about what to process. Just telling `pandas` to process something isn't good enough. Here's an example of working with the `Values.xls` file.

```
import pandas as pd
```

```
xls = pd.ExcelFile("Values.xls")
```

```
trig_values = xls.parse('Sheet1', index_col=None,
```

```
na_values=[ 'NA' ])
```

```
print(trig_values)
```

The code begins by importing the `pandas` library as normal. It then creates a pointer to the Excel file using the `ExcelFile()` constructor. This pointer, `xls`, lets you access a worksheet, define an index column, and specify how to present empty values. The index column is the one that the worksheet uses to index the records. Using a value of `None` means that pandas should generate an index for you. The `parse()` method obtains the values you request. You can read more about the Excel parser options at <https://pandas.pydata.org/pandas-docs/stable/generated/pandas.ExcelFile.parse.html>.

**TIP**

You don't absolutely have to use the two-step process of obtaining a file pointer and then parsing the content. You can also perform the task using a single step, like this: `trig_values = pd.read_excel("Values.xls", 'Sheet1', index_col=None, na_values=[ 'NA' ])`. Because Excel files are more complex, using the two-step process is often more convenient and efficient because you don't have to reopen the file for each read of the data.

Working with Nontext Data

Nontext data files consist of a series of bits. The file doesn't separate the bits from each other in any way. You can't simply look into the file and see any structure because there isn't any to see. Unstructured file formats rely on the file user to know how to interpret the data. For example, each pixel of a picture file could consist of three 32-bit fields. Knowing that each field is 32-bits is up to you. A header at the begin-

ning of the file may provide clues about interpreting the file, but even so, it's up to you to know how to interact with the file.

The example in this section shows how to work with a picture as an unstructured file. The example image is a public domain offering from https://commons.wikimedia.org/wiki/Main_Page. To work with images, you need to access the Scikit-image library (<https://scikit-image.org>), which is a free-of-charge collection of algorithms used for image processing. You can find a tutorial for this library at <http://scipy-lectures.github.io/packages/scikit-image/>. The first task is to be able to display the image onscreen using the following code. (This code can require a little time to run. The image is ready when the busy indicator disappears from the Notebook tab. You'll also see a `YAMLLoadWarning` message, which is a function of the library and not the book code.)

```
from skimage.io import imread
```

```
from skimage.transform import resize
```

```
from matplotlib import pyplot as plt
```

```
import matplotlib.cm as cm
```

```
example_file = ("http://upload.wikimedia.org/" +
```

```
"wikipedia/commons/7/7d/Dog_face.png")
```

```
image = imread(example_file, as_gray=True)
```

```
plt.imshow(image, cmap=cm.gray)
```

```
plt.show()
```

The code begins by importing a number of libraries. It then creates a string that points to the example file online and places it in `example_file`. This string is part of the `imread()` method call, along with `as_gray`, which is set to `True`. The `as_gray` argument tells Python to turn any color images into gray scale. Any images that are already in gray scale remain that way.

Now that you have an image loaded, it's time to render it (make it ready to display onscreen). The `imshow()` function performs the rendering and uses a grayscale color map. The `show()` function actually displays `image` for you, as shown in [Figure 4-9](#).

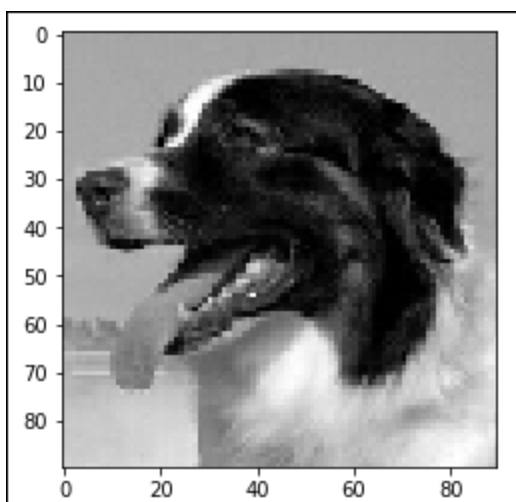


FIGURE 4-9: The image appears onscreen after you render and show it.

You now have an image in memory, and you may want to find out more about it. When you run the following code, you discover the image type and size:

```
print("data type: %s, shape: %s" %  
  
      (type(image), image.shape))
```

The output from this call tells you that the image type is a `numpy.ndarray` and the image size is 90 pixels by 90 pixels. The image is actually an array of pixels that you can manipulate in various ways.

For example, if you want to crop the image, you can use the following code to manipulate the image array:

```
image2 = image[5:70,0:70]
```

```
plt.imshow(image2, cmap=cm.gray)
```

```
plt.show()
```

The `numpy.ndarray` in `image2` is smaller than the one in `image`, so the output is smaller as well. [Figure 4-10](#) shows typical results. The purpose of cropping the image is to make it a specific size. Both images must be the same size for you to analyze them. Cropping is one way to ensure that the images are the correct size for analysis.

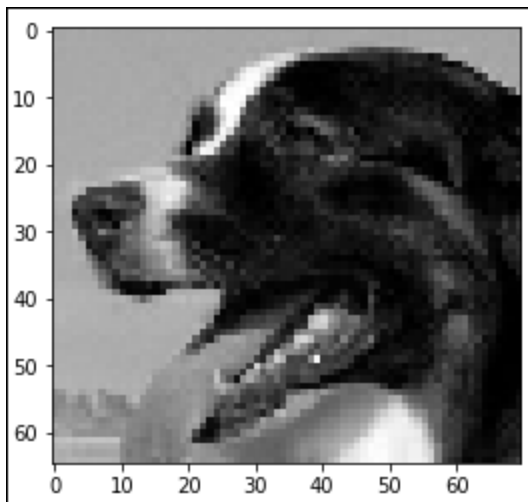


FIGURE 4-10: Cropping the image makes it smaller.

Another method that you can use to change the image size is to resize it. The following code resizes the image to a specific size for analysis (you can safely ignore the `UserWarning` message that displays when you run this code):

```
image3 = resize(image2, (30, 30), mode='symmetric')
```

```
plt.imshow(image3, cmap=cm.gray)
```

```
print("data type: %s, shape: %s" %  
  
(type(image3), image3.shape))
```

The output from the `print()` function tells you that the image is now 30 pixels by 30 pixels in size. You can compare it to any image with the same dimensions.

After you have all the images the right size, you need to flatten them. A dataset row is always a single dimension, not two dimensions. The image is currently an array of 30 pixels by 30 pixels, so you can't make it part of a dataset. The following code flattens `image3` so that it becomes an array of 900 elements that is stored in `image_row`.

```
image_row = image3.flatten()  
  
print("data type: %s, shape: %s" %  
  
(type(image_row), image_row.shape))
```

Notice that the type is still a `numpy.ndarray`. You can add this array to a dataset and then use the dataset for analysis purposes. The size is 900 elements, as anticipated.

Downloading Online Datasets

You're unlikely to find enough local data resources to fulfill every analysis need. At some point, you need to locate data online to perform experimentation or develop new analysis methods. Fortunately, you don't have to look far. The following sections discuss how to acquire and use online datasets.

Working with package datasets

Most languages support datasets you can use to perform simple data tasks or experimentation without having to create a dataset of your own. For example, Python supports a number of these datasets in Python packages such as the Scikit-learn library. You can see a list of them at <https://scikit-learn.org/stable/datasets/index.html>. When working with R, many of these datasets are included directly with the language, as shown at <https://stat.ethz.ch/R-manual/R-devel/library/datasets/html/00Index.html>. These datasets demonstrate various ways in which you can interact with data, and you use them in the examples to perform a variety of tasks. The following list provides a quick overview of the functions used to import the datasets from Scikit-learn into your Python code:

- `load_boston()` : Regression analysis with the Boston house prices dataset
- `load_iris()` : Classification with the Iris dataset
- `load_digits([n_class])` : Classification with the digits dataset
- `fetch_20newsgroups( subset='train' )` : Data from 20 newsgroups

The technique for loading each of these datasets is the same across examples. The following example shows how to load the Boston house prices dataset:

```
from sklearn.datasets import load_boston
```

```
Boston = load_boston()
```

```
print(Boston.data.shape)
```

The output from the `print()` call is `(506, 13)`.



TIP

Package datasets need not necessarily be contrived. Keras offers a downloadable wrapper for IMDb data. This dataset appears among other useful datasets at <https://keras.io/datasets/>. In particular, the IMDb textual data offered by Keras is cleansed of punctua-

tion, normalized into lowercase, and transformed into numeric values. Each word is coded into a number representing its ranking in frequency. Most frequent words have low numbers; less frequent words have higher numbers. Using the Keras datasets is similar to using other packages, as shown here:

```
from keras.datasets import imdb  
  
top_words = 10000  
  
(x_train, y_train),  
(x_test, y_test) = imdb.load_data(num_words=top_words,  
                                   seed=21)  
  
print("Training examples: %i" % len(x_train))  
  
print("Test examples: %i" % len(x_test))
```

Using public domain datasets

Public domain datasets enable you to expand the number of datasets at your disposal without having to generate the data yourself. In many cases, these public domain datasets are more complex than their package counterparts, but they also model the real world better. The following sections discuss how you can work with public domain datasets.

Downloading the file

Before you can use a public domain dataset, you need to locate the one that best matches your analysis scenario, which can take time (albeit less time than creating a dataset of your own unless the dataset you

need is quite simple). Fortunately, you can find repositories and compilations of datasets online, such as the ones found at Awesome Public Datasets (<https://github.com/awesomedata/awesome-public-datasets>) and Data.gov (<https://catalog.data.gov/dataset?q=public+domain>). You should also spend time looking at articles for potential dataset sources, such as this one at *Forbes* (<https://www.forbes.com/sites/bernardmarr/2016/02/12/big-data-35-brilliant-and-free-data-sources-for-2016/#54573865b54d>).

Some of these datasets come from the scientific community. For example, Kaggle (<https://www.kaggle.com/>) is a huge community of data scientists and others who need to work with large datasets to obtain the information needed to meet various goals. You can create new projects on Kaggle, view the work done by others on completed projects, or participate in one of its ongoing competitions. However, Kaggle is more than simply a community of really smart people who like to play with data; it's also a place where you can obtain resources needed to learn all about deep learning and to create projects of your own.

**TIP**

The best place to find out how Kaggle can help you discover more about deep learning is at

<https://www.kaggle.com/m2skills/datasets-and-tutorial-kernels-for-beginners> . This site lists the various datasets and tutorial kernels that Kaggle provides. A *tutorial kernel* is a kind of project you use to learn how to analyze data in various ways. For example, you can find a tutorial kernel about mushroom classification at <https://www.kaggle.com/uciml/mushroom-classification> .

Opening and using an archive

Seeing an example of how to work with public domain data is useful. This section relies on an example of a public domain downloadable file — the German Traffic Sign Recognition Benchmark (GTSRB) found at this Institute für NeuroInformatik at Ruhr-Universität Bochum page: <http://benchmark.ini.rub.de/?section=gtsrb>. The following code snippet downloads it to the same directory as the Python code. Note that the download process can take a little time to complete, so now might be a good time to refill your teacup.

```
import urllib.request

url = "https://sid.erda.dk/public/archives/\
daaeac0d7ce1152aea9b61d9f1e19370/\
GTSRB_Final_Training_Images.zip"

filename = "./GTSRB_Final_Training_Images.zip"

urllib.request.urlretrieve(url, filename)
```

After retrieving the dataset as a `.zip` file from the Internet, the code sets an image size. (All images are resized to square images, so the size represents the sides in pixels.) The code also sets the portion of data to keep for testing purposes, which means excluding certain images from training to have a more reliable measure of how the neural network works.

A loop through the files stored in the downloaded `.zip` file retrieves individual images, resizes them, stores the class labels, and appends the images to two separate lists: one for the training and one for testing purposes. The sorting uses a hash function, which translates the image name into a number and, based on that number, decides where to append the image.

```
import zipfile
```



```
from imageio import imread

from skimage.transform import resize

IMG_SIZE = 32

TEST_SIZE = 0.2

X, Xt, y, yt = list(), list(), list(), list()

archive = zipfile.ZipFile(

    './GTSRB_Final_Training_Images.zip', 'r')

file_paths = [file for file in archive.namelist()

    if '.ppm' in file]

for filename in file_paths:

    with archive.open(filename) as img_file:

        img = imread(img_file.read())

        img = resize(img,

            output_shape=(IMG_SIZE, IMG_SIZE),

            mode='reflect', anti_aliasing=True)

        img_class = int(filename.split('/')[-2])
```

```
if (hash(filename) % 1000) / 1000 > TEST_SIZE:
```

```
    X.append(img)
```

```
    y.append(img_class)
```

```
else:
```

```
    Xt.append(img)
```

```
    yt.append(img_class)
```

```
archive.close()
```

At this point, you could use the data for analysis purposes of some sort. That is, you didn't have to generate the data yourself; someone else obtained it for you. So you can focus on your code without having to consider a data source until your code is ready for use in the real world.

Table of contents collapsed